Detecting Port Scans with Machine Learning

DeepNetSec-2026 — Module 01

Submitted by: Pedapati Sujith
Program: B.Tech Electronics and Communication Engineering
Institute: IIITDM Kurnool
Date: June 22, 2026

1 Introduction

A port scan is a reconnaissance technique in which an attacker probes multiple destination ports to identify available services on a target system. The objective of this project was to build machine learning models that classify network traffic flows as normal traffic or port-scan traffic.

The labels used were:

- 0: Normal traffic
- 1: Port-scan traffic

2 Dataset Preparation

PCAPNG network captures were converted into flow-based CSV data. Each row represented one bidirectional network flow.

Identity-related columns such as IP addresses, MAC addresses, timestamps, device identifiers, VLAN identifiers, tunnel identifiers, and application text metadata were removed. These columns can cause data leakage because the model may memorize the Kali machine identity instead of learning actual port-scan behaviour.

The final dataset retained behavioural features including destination port, protocol, IP version, flow duration, packet counts, byte counts, and traffic direction statistics.

Table 1: Important Behavioural Features

Feature	Purpose
Destination port	Port scans probe many destination ports.
Protocol	TCP and UDP traffic may have different scan patterns.
Flow duration	Scan flows are often short.
Packet counts	Scan flows often contain fewer packets.
Byte counts	Scan flows often have smaller payload sizes.
Traffic direction	Scan attempts may receive limited reply traffic.

3 Class Balancing and Exploratory Data Analysis

The original dataset contained 70,339 port-scan flows and only 58 normal flows. This was highly imbalanced.

Random undersampling was applied to the scan class. The final balanced dataset contained:

- 58 normal flows
- 58 port-scan flows

Therefore, the final dataset contained 116 total samples.

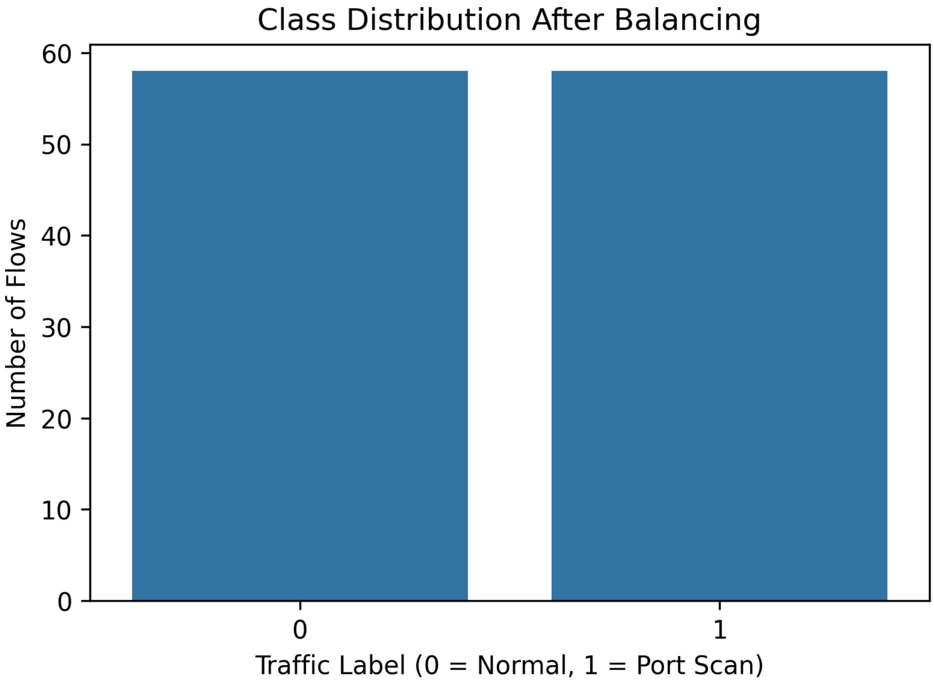

Figure 1: Class distribution after random undersampling. Both normal and port-scan classes contain 58 flows.

The class-distribution plot confirms that the data was balanced before model training. This prevents the model from becoming biased toward the majority scan class.

Destination-port distribution was analysed because port scans probe multiple services and ports. Normal traffic generally communicates with a smaller set of destination ports.

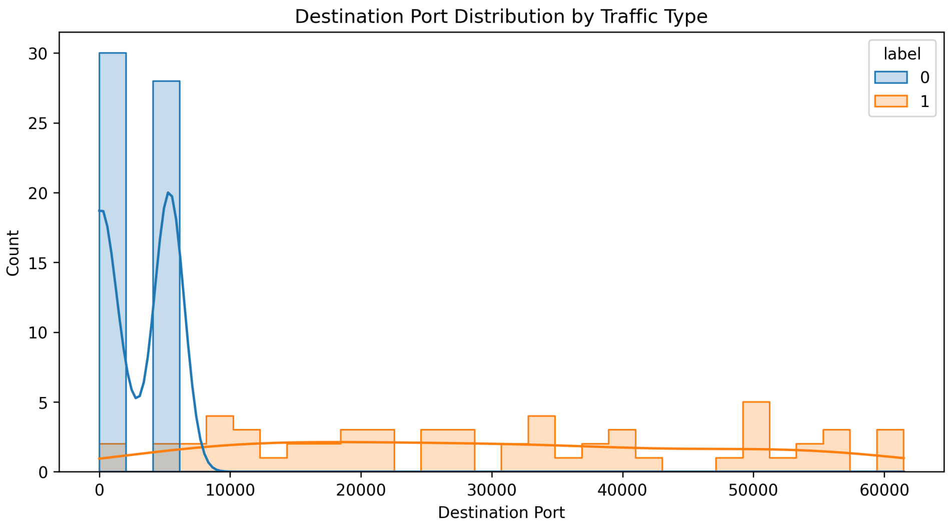

Figure 2: Destination port distribution for normal and port-scan traffic.

The destination-port plot demonstrates an important scan signature: scan traffic is spread across a wider range of ports because the attacker attempts to discover open services.

Packet count was also analysed because port scans often involve short interactions and fewer exchanged packets than normal communication.

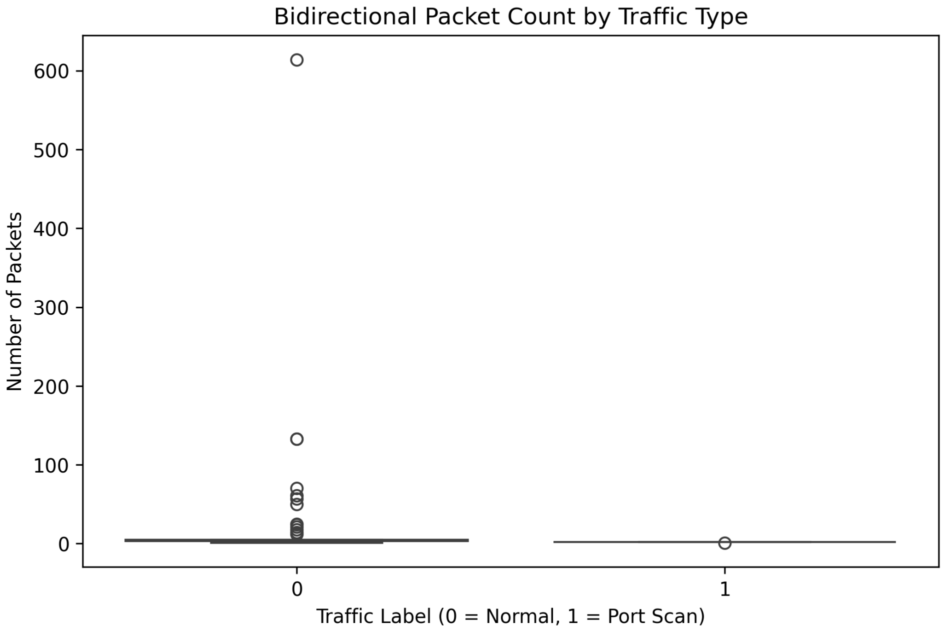

Figure 3: Bidirectional packet count comparison between normal and port-scan flows.

The packet-count comparison shows that flow-level packet behaviour can help distinguish normal communication from scan attempts.

4 Preprocessing

The target column was already numeric, where 0 represented normal traffic and 1 represented port-scan traffic. Therefore, Label Encoding was not required.

The dataset was divided into feature matrix X and target vector y. An 80:20 stratified train-test split was used, producing approximately 92 training samples and 24 testing samples.

StandardScaler was used for Logistic Regression because it is sensitive to feature scale. Random Forest was trained using unscaled data because tree-based models do not require scaling.

5 Models

5.1 Logistic Regression

Logistic Regression was used as a baseline model. It was trained using scaled features and compared with the Random Forest classifier.

5.2 Random Forest

Random Forest was used as the main model because it can capture non-linear relationships between network-flow features.

The model configuration was:

- Number of trees: 100
- Maximum depth: 5
- Minimum samples per leaf: 2
- Random state: 42

6 Evaluation Results

The models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices. Recall for port scans is important because a false negative means that a real scan is incorrectly classified as normal traffic.

6.1 Logistic Regression Performance

Below is the confusion matrix heatmap for the baseline Logistic Regression model evaluated on the test set:

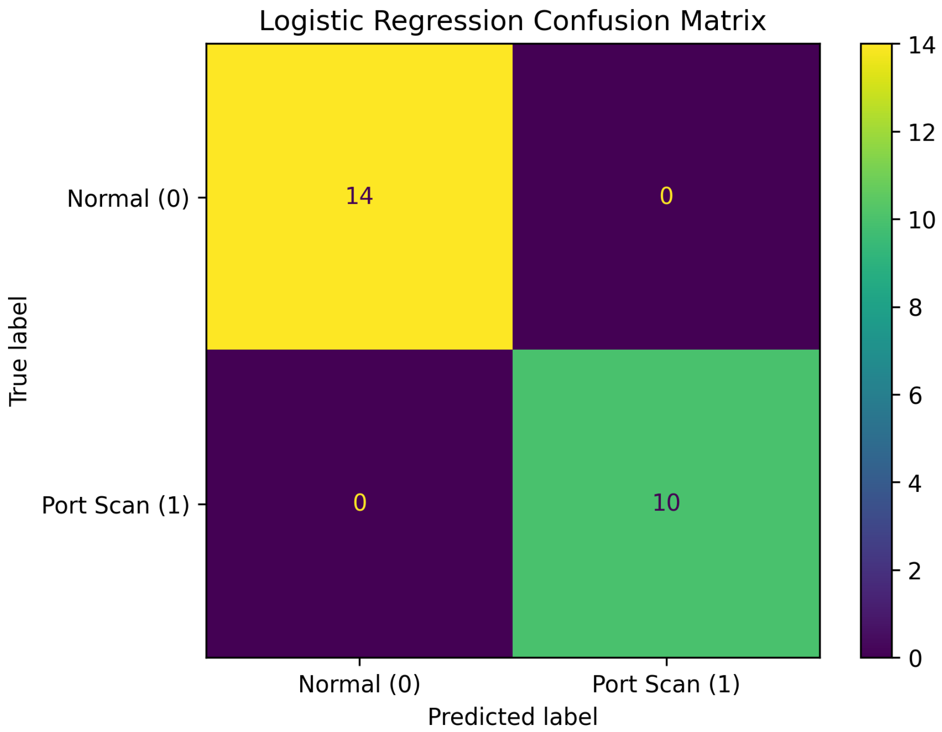

Figure 4: Logistic Regression confusion matrix heatmap.

6.2 Random Forest Performance

The Random Forest model achieved the following results on the held-out test set:

Table 2: Random Forest Performance

Class	Precision	Recall	F1-score	Support
Normal (0)	1.00	1.00	1.00	14
Port Scan (1)	1.00	1.00	1.00	10
Accuracy	1.00 on 24 test flows			

The model correctly classified all normal and port-scan flows in the held-out test set. There were no false positives and no false negatives.

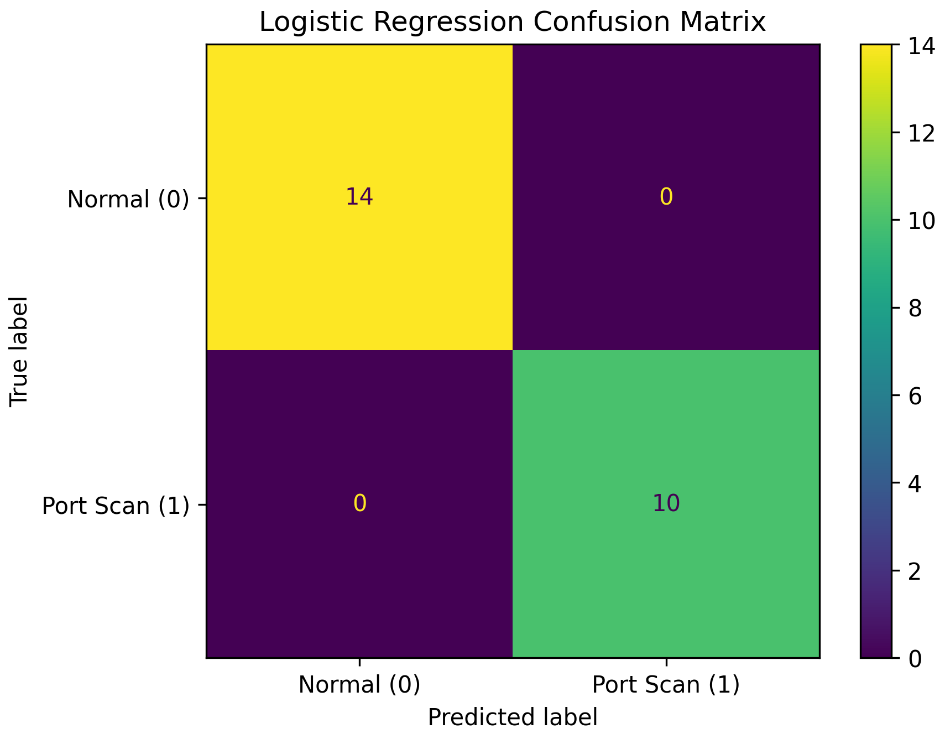

Figure 5: Random Forest confusion matrix heatmap on the held-out test set.

Five-fold stratified cross-validation produced the following F1-scores:

$$[1.00,\ 1.00,\ 0.9167,\ 1.00,\ 1.00]$$

The mean F1-score was 0.9833 and the standard deviation was 0.0333. This indicates stable performance across different folds.

7 Why IP and MAC Addresses Were Removed

IP and MAC addresses were removed to prevent data leakage. If these columns were kept, the model could memorize that traffic from a particular Kali IP address represents a scan. Such a model may perform well only in the same lab environment and fail when the attacker or target changes.

Removing identity columns forces the model to learn behavioural features such as destination port, packet count, byte count, duration, and traffic direction.

8 Conclusion

This project implemented a complete machine learning pipeline for port-scan detection. The data was cleaned, balanced, explored, and used to train Logistic Regression and Random Forest models.

Random Forest achieved 100% accuracy on the held-out test set and a mean cross-validation F1-score of 0.9833. However, the dataset contained only 116 balanced samples, so more normal traffic data would be needed for stronger real-world generalization.